CommonVoice

Domain-Driven Design — Architecture Guide

Clean Architecture · .NET 10 · Phil-Izra

This document explains the domain model of the CommonVoice platform in plain English — what each piece is, why it exists, and how everything connects. No prior knowledge of Domain-Driven Design (DDD) is assumed.

1. WHAT IS DOMAIN-DRIVEN DESIGN (DDD)?

Domain-Driven Design is a way of building software where the code is organised around the real-world problem it solves — not around databases, screens, or frameworks. In DDD, the language of the business (the "domain") is baked directly into the code.

Why CommonVoice needs DDD

The CommonVoice platform has real, complex rules:

- A protest can only be activated if it has at least one demand.
- A participant can only check in once per device per protest.
- The Grievance Weight Score (GWS) is calculated using a specific four-component formula.
- The nightly job computes a province-level weighted average across all protests.

These rules belong in the core of the application — not scattered across database queries or controller methods. DDD ensures these rules live in one place and are always enforced, no matter how the application grows.

The Three Golden Rules of DDD Used Here

- Aggregates own their data — nothing outside an aggregate can change its state directly.
- Domain Events communicate what just happened — other parts of the system react to them.
- Repository interfaces sit in the domain — the database implementation is kept out of sight.

2. THE BUILDING BLOCKS

Entity

An Entity is an object that has a unique identity (an Id) that never changes, even if its other properties change. Two entities are equal if they share the same Id — not if they have the same data.

```
public abstract class Entity
{
    public Guid Id { get; protected set; } = Guid.NewGuid();

    // Every entity can raise domain events
    private readonly List<IDomainEvent> _domainEvents = [];
    protected void Raise(IDomainEvent e) => _domainEvents.Add(e);
}
```

Aggregate Root

An Aggregate Root is a special kind of entity that acts as the "front door" to a cluster of related objects. All changes to the cluster must go through the root — you never reach inside and change a child directly.

In CommonVoice the four aggregate roots are: User, Protest, Participant, and GrievanceScore.

Value Object

A Value Object has no identity. Two value objects are equal if their data is equal. They are also immutable — you never change them, you replace them. They are ideal for enforcing rules about a concept.

- Province — must be one of South Africa's nine provinces.
- GrievanceCategory — must be one of the 11 defined categories.
- GeoPoint — a latitude/longitude pair with built-in Haversine distance calculation.
- GwsScore — bundles the four GWS components and computes the final score.

Domain Event

A Domain Event is a record of something that happened. It is raised inside an aggregate and published outwards so that other parts of the system (like background jobs or notification services) can react without the aggregate needing to know about them.

```
public interface IDomainEvent
{
    DateTime OccurredAt { get; }
}
```

3. VALUE OBJECTS IN DETAIL

Province

Wraps a province name and rejects any value that is not one of South Africa's nine recognised provinces. This means it is impossible to create a protest in a province that does not exist — the error is caught at the domain level, before anything reaches the database.

```
var province = new Province("gauteng");       // OK
var province = new Province("atlantis");      // Throws ArgumentException
```

GrievanceCategory

Enforces that every protest is tagged with one of the 11 recognised grievance types:

```
"housing"           "unemployment"      "load_shedding"
"water_access"      "education"         "healthcare"
"safety_crime"      "infrastructure"    "labour_rights"
"illegal_immigration"                   "other"
```

GeoPoint

Stores a latitude/longitude pair and includes a Haversine method that calculates the straight-line distance (in metres) between two points on the Earth's surface. This is used to determine whether a participant is physically inside the protest boundary.

GwsScore (Grievance Weight Score)

Holds the four input components of the GWS formula and computes the final score:

```
GWS = (Reach × 0.35) + (Recurrence × 0.30) + (CrossSector × 0.20) + (Freshness × 0.15)

  Reach        — How many people participated (normalised to 0–100)
  Recurrence   — How often this grievance recurs across months
  CrossSector  — How many different sectors raised the same grievance
  Freshness    — How recent the protest activity is
```

All four components are validated to be between 0 and 100 before the object is created.

4. AGGREGATE ROOTS IN DETAIL

User · roles: Admin · User · Journalist

The User aggregate represents every person who has an account on the platform. The Role property determines what they can do:

Admin | User (Organiser) | Journalist

- Admin — manages the platform, can verify organisers, moderate content.
- User — a protest organiser. Must supply a Sector and Province on registration, because their protests are tied to a specific area and sector.
- Journalist — read-only access. Can embed the public GWS priority graph and access protest data for reporting.

Sector and Province are nullable on the User model because Admins and Journalists do not organise protests and have no need for those fields.

```
// Only organisers must have sector + province
if (role == UserRole.User && (sector is null || province is null))
    throw new ArgumentException("Organisers must supply a sector and province");
```

Protest

The Protest aggregate is the heart of the platform. It models a real-world civic protest from creation through to completion.

Lifecycle (Status Machine)

```
Draft  !’  Scheduled  !’  Active  !’  Completed
                   !˜  Cancelled
```

- Draft / Scheduled — the organiser is building the protest. Demands can be added.
- Active — the protest is live. A one-time join token (valid for 12 hours) is generated so participants can check in via QR code or link.
- Completed — the protest has ended. The join token is cleared. The nightly job will now include this protest in the GWS calculation.
- Cancelled — the protest was called off.

Key Rules Enforced by the Aggregate

- A protest cannot be activated without at least one demand. The organiser must state what they are marching for.
- The gathering radius must be between 100 m and 10,000 m.
- The scheduled date must be in the future.
- A demand's response status can only be: ignored, acknowledged, in_progress, or resolved.

Demand — Child Entity of Protest

A Demand is a specific grievance raised during a protest (e.g. "Fix the water pipes on Mandela

Street"). It is a child entity — it belongs entirely to one Protest and cannot exist without it. Government or officials can later update its ResponseStatus to track accountability.

Participant

A Participant is an anonymous check-in. The person does not need an account — they scan the protest's QR code and the app records their device fingerprint and GPS coordinates.

- IsWithinBoundary is set to true if the GPS location falls within the protest's radius. This prevents remote or fraudulent check-ins from inflating the count.
- The IParticipantRepository.ExistsAsync check prevents the same device from checking in twice to the same protest.

GrievanceScore

Created exclusively by the nightly background job — never by user action. Each record captures the GWS score for one protest in a specific month and year. The ProvincialGwsCalculator service then aggregates all scores for the same province and grievance category into a single provincial GWS.

```
// Marcher-weighted average across all protests
ProvincialGWS = :2‡  'F–6—  çD6÷VçB r uu2’ r £(participantCount)
```

This weighting ensures that a protest with 10,000 participants has more influence on the provincial score than one with 50.

5. DOMAIN EVENTS

Domain Events are raised inside aggregates and dispatched after the database transaction commits. They decouple the aggregate from downstream concerns like emails, notifications, and score recalculation.

Event	Raised by	What happens next
`UserRegisteredEvent`	User	Send welcome / verification email
`ProtestCreatedEvent`	Protest	Notify admins of new protest
`ProtestActivatedEvent`	Protest	Generate QR code, open check-in window
`ProtestCompletedEvent`	Protest	Trigger nightly GWS recalculation
`ParticipantCheckedInEvent`	Participant	Update live crowd count display

6. REPOSITORY INTERFACES

Repositories are the only way the application layer talks to the database. The interfaces are defined in the Domain layer — this means the domain has zero knowledge of Entity Framework, PostgreSQL, or any storage technology. The actual implementations live in the Infrastructure layer.

Interface	Responsibility
`IUserRepository`	Find users by Id or email, persist new users
`IProtestRepository`	Find protests by Id, join token, province+category+month, persist protests
`IParticipantRepository`	Check for duplicate check-ins, count participants per protest
`IGrievanceScoreRepository`	Load scores for provincial aggregation, upsert nightly results

7. JWT AUTHENTICATION MODELS

Authentication uses JSON Web Tokens (JWT). A user logs in with email and password and receives a signed token they attach to every future request. The token encodes their UserId and Role so the API knows who they are without hitting the database on every call.

JwtSettings (appsettings.json !’ "Jwt" section)

```
{
  "Jwt": {
    "Secret":        "a-long-random-string-only-the-server-knows",
    "Issuer":        "CommonVoice.API",
    "Audience":      "CommonVoice.Client",
    "ExpiryMinutes": 60
  }
}
```

LoginRequest

Email — must be a valid email address.

- Password — minimum 8 characters.

RegisterRequest

- Name, Email, Password — same rules as login.
- Role — one of Admin, User, or Journalist.
- Sector and Province — required only when Role is User (organiser).

AuthResponse (returned on successful login or register)

```
{
  "token":     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresAt": "2026-06-16T14:30:00Z",
  "userId":    "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name":      "Sipho Dlamini",
  "role":      "User"
}
```

8. FOLDER STRUCTURE

Everything currently lives in one project (CommonVoice.API). The folder layout mirrors the layers of Clean Architecture so that, when the project grows, each folder can be extracted into its own .NET class library with minimal friction.

```
CommonVoice.API/
% % %  Domain/
%    % % %  Shared/                  !• Entity, AggregateRoot, IDomainEvent
%    % % %  ValueObjects/            !• Province, GrievanceCategory, GeoPoint, GwsScore
%    % % %  Aggregates/
%    %    % % %  Users/               !• User, UserRole
%    %    % % %  Protests/            !• Protest, Demand, ProtestEvents
%    %    % % %  Participants/        !• Participant, ParticipantCheckedInEvent
%    %    % % %  GrievanceScores/     !• GrievanceScore
%    % % %  Services/                !• ProvincialGwsCalculator
%    % % %  Repositories/            !• IUserRepository, IProtestRepository, ...
% % %  Models/
%    % % %  Auth/                    !• JwtSettings, LoginRequest, RegisterRequest, AuthResponse
% % %  Controllers/                !• (to be added: AuthController, ProtestController, ...)
% % %  Program.cs                   !• JWT wiring, service registration
% % %  appsettings.json             !• Jwt config section
```

9. HOW IT ALL FITS TOGETHER — AN END-TO-END STORY

Here is the full journey of a protest on CommonVoice, from registration to the public priority graph:

1 Organiser registers

A community leader signs up via POST /auth/register with Role = User, Sector = "civic", Province = "gauteng". The User aggregate validates the input, hashes the password (done in the Application layer), and raises a UserRegisteredEvent. A JWT token is returned.

2 Protest is created

The organiser calls POST /protests. The Protest aggregate is created in Draft status. They call POST /protests/{id}/demands to add their grievances (e.g. "Fix the potholes on Main Road"). Status is still Draft.

3 Protest goes live

The organiser calls POST /protests/{id}/activate. The aggregate checks that at least one demand exists, flips status to Active, generates a 16-character join token, and raises a ProtestActivatedEvent. The Application layer handles this event to generate a QR code for sharing.

4 Participants check in

Attendees scan the QR code. Each check-in calls POST /protests/{id}/checkin with the device fingerprint and GPS coordinates. The Participant aggregate calculates whether the person is within the boundary using GeoPoint.DistanceMetresTo(). Duplicate check-ins are blocked by IParticipantRepository.ExistsAsync.

5 Protest is completed

At the end of the day, the organiser calls POST /protests/{id}/complete. Status becomes Completed, the join token is cleared, and ProtestCompletedEvent is raised.

6 Nightly GWS calculation

A background job runs at midnight. For each province + grievance category combination, it loads all completed protests for the month, computes GwsScore per protest, saves each GrievanceScore, then calls ProvincialGwsCalculator.Calculate() to get the weighted provincial score.

7 Public priority graph

Journalists and government officials see the aggregated GWS scores on the public dashboard — ranked by province and grievance. They can embed the live graph or cite the score in reports. The data is verifiable and tamper-resistant because every participant check-in is individually recorded.

10. QUICK GLOSSARY

Aggregate	A cluster of objects treated as one unit. Changes go through the root.
Aggregate Root	The "front door" of an aggregate. The only object the outside world holds a reference to.
Bounded Context	A boundary within which a specific model and language applies.
Domain	The real-world subject area the software is solving — civic protest data in our case.
Domain Event	A record of something important that happened inside the domain.
Entity	An object with a unique Id that persists over time.
GWS	Grievance Weight Score — a composite metric combining reach, recurrence, cross-sector breadth, and freshness.
Invariant	A rule that must always be true inside an aggregate (e.g. a protest must have at least one demand to go live).
Repository	An interface that abstracts data storage. The domain defines it; Infrastructure implements it.
Value Object	An immutable object defined by its data, not by an Id (e.g. Province, GeoPoint).